

Proyecto 3: Tabla de Productos Dinámica AJAX (XMLHttpRequest)

1. Objetivos del proyecto

- Hacer **peticiones GET dinámicas** usando `XMLHttpRequest` según la entrada del usuario.
- Mostrar los resultados en una **tabla Bootstrap**.
- Permitir **filtrar y ordenar** productos sin recargar la página.
- Integrar **Bootstrap** y **AJAX** clásico para manipulación del DOM.

Lo que se espera que haga el alumno:

1. Crear un servidor que devuelva productos en JSON y acepte filtros.
2. Crear un formulario de filtro (categoría y precio máximo).
3. Mostrar los resultados en una tabla HTML.
4. Añadir botones para ordenar por precio ascendente/descendente.

2. Servidor Node.js con Express

```
// server.js
const express = require('express');
const cors = require('cors');
const app = express();

app.use(cors());

// Datos de ejemplo
const productos = [
  { id: 1, nombre: "Laptop", categoria: "Electrónica", precio: 1200 },
  { id: 2, nombre: "Mouse", categoria: "Electrónica", precio: 25 },
  { id: 3, nombre: "Teclado", categoria: "Electrónica", precio: 45 },
  { id: 4, nombre: "Leche", categoria: "Alimentos", precio: 2 },
  { id: 5, nombre: "Pan", categoria: "Alimentos", precio: 1.5 },
  { id: 6, nombre: "Camisa", categoria: "Ropa", precio: 35 },
  { id: 7, nombre: "Pantalón", categoria: "Ropa", precio: 50 }
];

// Ruta GET con filtros opcionales
app.get('/api/productos', (req, res) => {
  let { categoria, precioMax } = req.query;
  let resultado = productos;

  if (categoria) {
    resultado = resultado.filter(p => p.categoria.toLowerCase() === categoria.toLowerCase());
  }
});
```

```

    }

    if (precioMax) {
        const max = parseFloat(precioMax);
        resultado = resultado.filter(p => p.precio <= max);
    }

    res.json(resultado);
});

// Iniciar servidor
const PORT = 3002;
app.listen(PORT, () => {
    console.log(`Servidor escuchando en http://localhost:${PORT}`);
});

```

- **Puerto: 3002**
- **Ejemplos de URL:**
 - **Todos** → GET `http://localhost:3002/api/productos`
 - **Por categoría** → GET `http://localhost:3002/api/productos?categoria=Electrónica`
 - **Por categoría + precio** → GET `http://localhost:3002/api/productos?categoria=Ropa&precioMax=40`

3. Cliente HTML + Bootstrap

```

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Productos Dinámicos AJAX</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.
min.css" rel="stylesheet">
</head>
<body class="container py-4">

    <h1>Filtrar Productos</h1>

    <div class="row mb-3">
        <div class="col-md-4">
            <input type="text" id="categoria" class="form-control"
placeholder="Categoría">
        </div>
        <div class="col-md-4">
            <input type="number" id="precioMax" class="form-control"
placeholder="Precio máximo">
        </div>
        <div class="col-md-4">
            <button id="btnFiltrar" class="btn btn-
primary">Filtrar</button>
            <button id="btnCargarTodos" class="btn btn-
secondary">Cargar Todos</button>
            <button id="btnOrdenAsc" class="btn btn-success">Orden
Asc</button>

```

```

        <button id="btnOrdenDesc" class="btn btn-danger">Orden
Desc</button>
    </div>
</div>

<table class="table table-striped">
    <thead>
        <tr>
            <th>ID</th>
            <th>Nombre</th>
            <th>Categoría</th>
            <th>Precio</th>
        </tr>
    </thead>
    <tbody id="tablaProductos"></tbody>
</table>

<script src="script.js"></script>
</body>
</html>

```

4. JavaScript (script.js) con XMLHttpRequest

```

const tabla = document.getElementById('tablaProductos');
const btnCargarTodos = document.getElementById('btnCargarTodos');
const btnFiltrar = document.getElementById('btnFiltrar');
const btnOrdenAsc = document.getElementById('btnOrdenAsc');
const btnOrdenDesc = document.getElementById('btnOrdenDesc');
const inputCategoria = document.getElementById('categoria');
const inputPrecio = document.getElementById('precioMax');

const API_BASE = 'http://localhost:3002/api/productos';
let productosGlobal = [];

// Mostrar productos en tabla
function mostrarProductos(productos) {
    tabla.innerHTML = '';
    productos.forEach(p => {
        const row = document.createElement('tr');
        row.innerHTML = `
            <td>${p.id}</td>
            <td>${p.nombre}</td>
            <td>${p.categoria}</td>
            <td>${p.precio}</td>
        `;
        tabla.appendChild(row);
    });
}

// Función para hacer GET con XMLHttpRequest
function ajaxGet(url, callback) {
    const xhr = new XMLHttpRequest();
    xhr.open('GET', url, true);
    xhr.onload = function() {
        if (xhr.status === 200) {
            const data = JSON.parse(xhr.responseText);
            callback(data);
        } else {
            alert('Error en la petición AJAX');
        }
    };
}

```

```

        }
    };
    xhr.onerror = () => alert('Error en la petición AJAX');
    xhr.send();
}

// Cargar todos los productos
btnCargarTodos.addEventListener('click', () => {
    ajaxGet(API_BASE, (data) => {
        productosGlobal = data;
        mostrarProductos(data);
    });
});

// Filtrar productos
btnFiltrar.addEventListener('click', () => {
    let url = API_BASE + '?';
    const categoria = inputCategoria.value.trim();
    const precioMax = inputPrecio.value.trim();
    if(categoria) url +=
`categoria=${encodeURIComponent(categoria)}&`;
    if(precioMax) url +=
`precioMax=${encodeURIComponent(precioMax)}&`;

    ajaxGet(url, (data) => {
        productosGlobal = data;
        mostrarProductos(data);
    });
});

// Orden Ascendente
btnOrdenAsc.addEventListener('click', () => {
    const sorted = [...productosGlobal].sort((a,b) => a.precio -
b.precio);
    mostrarProductos(sorted);
});

// Orden Descendente
btnOrdenDesc.addEventListener('click', () => {
    const sorted = [...productosGlobal].sort((a,b) => b.precio -
a.precio);
    mostrarProductos(sorted);
});

```

5. Funcionamiento del proyecto

1. Ejecutar el servidor:

```
node server.js
```

2. Abrir index.html en el navegador.
3. **Cargar todos los productos:** click en **Cargar Todos**.
4. **Filtrar productos:** ingresar categoría y/o precio máximo → click en **Filtrar**.
5. **Ordenar por precio:** usar **Orden Asc** o **Orden Desc**.
6. Todo se hace **sin recargar la página**, usando XMLHttpRequest.

6. Ejercicios propuestos para alumnos

1. Agregar un input para filtrar por **nombre de producto**.
2. Mostrar **mensaje “Cargando...”** mientras llega la respuesta AJAX.
3. Resaltar productos con **precio mayor a 100** usando Bootstrap (`table-danger`).
4. Implementar **paginación**: mostrar 3 productos por página con botones Next/Prev.
5. Combinar filtros: categoría + precio + nombre.
6. Guardar todos los productos filtrados en **localStorage** para mantenerlos visibles al recargar la página.